instituto superior de engenharia de lisboa

Departamento de Engenharia Informática

U!REKA - Play4Change: A Multiplatform Gamified Educational Platform for Sustainability and Digital Education

51620 : Radesh Ilesh Gamanbhai Govind (A51620@alunos.isel.pt)

Relatório para a Unidade Curricular de Licenciatura em Engenharia Informática e de Computadores da Licenciatura em Engenharia Informática e de Computadores

Professores : Professor Nuno Miguel Soares Datia
Professor António Serrador

21 – março – 2026

Resumo

U!REKA Play4Change é uma plataforma educacional gamificada e multiplataforma que combina jogos sérios com inteligência artificial para promover a aprendizagem em sustentabilidade e educação digital. A aplicação móvel, disponível em Android e iOS a partir de uma única base de código desenvolvida com Kotlin Multiplatform e Compose Multiplatform, apresenta aos jogadores tarefas diárias geradas automaticamente por um modelo de linguagem de grande dimensão através da biblioteca LangChain4j integrada com a API Mistral. As tarefas são produzidas em processos noturnos em lote e armazenadas numa base de dados PostgreSQL com extensão pgvector para desduplicação semântica, garantindo a diversidade do conteúdo. A autenticação é gerida através de OAuth 2.0 com rotação de tokens JWT. O motor de pontuação atribui recompensas e penalizações para incentivar o envolvimento dos utilizadores. O backend é implementado com Spring Boot e implantado via Docker Compose, suportando configuração flexível por domínio temático e nível de audiência.

Abstract

U!REKA Play4Change is a multiplatform gamified educational platform targeting sustainability and digital education, developed as a final year undergraduate project. Built with Kotlin Multiplatform and Compose Multiplatform, it delivers a unified Android and iOS experience from a single codebase, eliminating duplication of platform-specific logic. The platform presents players with AI-generated daily tasks produced by nightly batch jobs using LangChain4j and the Mistral language model API, enabling zero cold-start latency at runtime. Tasks are stored in a PostgreSQL database extended with pgvector for semantic deduplication, ensuring content novelty over time. Player identity is managed through OAuth 2.0 with JWT token rotation, providing secure and stateless authentication. A server-side scoring engine drives engagement through configurable rewards and penalties. The backend, built on Spring Boot and deployed via Docker Compose, supports flexible configuration by subject domain and audience level, making the platform reusable across different educational networks.

Índice

Lista de Figuras **ix**

Lista de Tabelas **xi**

Lista de Listagens **xiii**

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Objectives 1

1.3 Document Structure 2

2 State of the Art **3**

2.1 Gamification in Education 3

2.1.1 Evidence of Effectiveness 3

2.2 Serious Games 3

2.3 Multiplatform Mobile Development 4

2.3.1 Native versus Cross-Platform Approaches 4

2.3.2 Justification for Kotlin Multiplatform and Compose Multiplatform 4

2.4 AI-Generated Content in Education 4

2.4.1 Large Language Models for Educational Content 4

2.4.2 LangChain4j and the Mistral API 4

2.5 Related Platforms 5

3 Tutorial de `LaTeX` com exemplos **7**

3.1 Tabelas 7

3.2 Importar Figuras 7

3.2.1 Inserir figuras lado-a-lado com texto 7

3.2.2 Outras imagens 8

3.3 Listagens 8

Referências **9**

A Applied Survival Analysis by Hosmer and Lemeshow **i**

Lista de Figuras

3.1 Vectorial image 7
3.2 Subfigure example with vectorial and no-vectorial images 8

Lista de Tabelas

3.1 Table's rules. 7

Lista de Listagens

3.1 Operações elementares de uma Calculadora 8

Capítulo 1

Introduction

1.1 Context and Motivation

Education systems worldwide face a persistent challenge: sustaining learner engagement over time, particularly in domains such as sustainability and digital literacy that demand continuous effort rather than one-off study sessions. Traditional instructional approaches rely heavily on static content, which is costly to produce and rapidly becomes outdated. Gamified platforms have emerged as a promising response to this challenge, consistently demonstrating measurable improvements in learner engagement and knowledge retention [2]. However, most existing solutions depend on manually authored question banks, which limits their scalability and increases operational cost.

A second structural problem compounds the first: the fragmentation of mobile ecosystems. Developing and maintaining separate native codebases for Android and iOS doubles engineering effort and slows the iteration cycle for small teams. Cross-platform frameworks have matured sufficiently to address this, but few gamified educational platforms have adopted them systematically.

U!REKA Play4Change was designed to address both problems simultaneously. By combining serious game mechanics with an AI-powered content generation pipeline, the platform produces fresh, domain-specific tasks automatically on a nightly schedule, removing the manual authoring bottleneck. By targeting Android and iOS from a single Kotlin Multiplatform codebase, it eliminates platform duplication without sacrificing native user-interface fidelity. The result is a platform that scales content continuously while remaining economically viable for small development teams and educational institutions.

1.2 Objectives

The project pursued four core engineering objectives:

Multiplatform delivery from a single codebase, targeting Android and iOS using Kotlin Multiplatform and Compose Multiplatform, so that all business logic and user-interface components are shared across both platforms.

Adaptive AI-generated content, producing domain-specific educational tasks nightly via a large language model pipeline built on LangChain4j and the Mistral API, with semantic deduplication through pgvector to prevent content repetition.

Engagement loop engineering, implementing a server-side scoring engine that rewards correct and timely responses and applies penalties for inactivity or incorrect answers, driving sustained player motivation.

Cost efficiency through batch generation, pre-computing all daily tasks during off-peak hours so that runtime requests are served from a local cache, eliminating inference latency and minimising API costs during active play sessions.

1.3 Document Structure

The remainder of this report is organised as follows. Chapter 2 surveys the relevant literature and existing technologies, covering gamification in education, serious games, multiplatform mobile development, AI-generated content, and related platforms. Chapter 3 describes the system architecture, detailing the design decisions behind the mobile application, the content generation pipeline, the authentication mechanism, and the scoring engine. Chapter 4 presents the implementation, including selected code artefacts and integration details. Chapter 5 covers evaluation, reporting on functional testing, performance measurements, and a qualitative assessment of the platform. Chapter 6 draws conclusions and outlines directions for future work.

Capítulo 2

State of the Art

2.1 Gamification in Education

Gamification is defined as the application of game design elements in non-game contexts [1]. In educational settings, these elements typically include points, leaderboards, badges, progress bars, and challenge structures. The underlying motivation is to harness the intrinsic drivers of game engagement—competition, mastery, and reward—to sustain learner effort in contexts where motivation often wanes.

2.1.1 Evidence of Effectiveness

Empirical evidence on gamification's effectiveness in education has accumulated steadily. A systematic literature review of 24 empirical studies found positive effects in the majority of cases, with social comparison mechanisms such as leaderboards and peer challenges producing the most consistent engagement gains [2]. However, the same review noted important moderating factors: the type of gamification element, the duration of the intervention, and the demographic profile of learners all influence outcomes. These nuances informed the design of U!REKA Play4Change's scoring engine, which combines individual progression with configurable penalty and reward thresholds rather than relying solely on competitive ranking.

2.2 Serious Games

Serious games are digital games designed with a primary purpose beyond pure entertainment, typically targeting learning, training, or behaviour change. Unlike simple gamification, serious games situate the learner inside a coherent game world with narrative context, thus providing richer motivational scaffolding [6].

A meta-analysis of serious game research spanning a decade found statistically significant improvements in learning outcomes across multiple disciplines, including science, health, and civic education [6]. The analysis identified immediate feedback and clear goal structures as the two strongest predictors of learning gain. U!REKA Play4Change draws on these findings by structuring each daily task as a self-contained challenge with instant scoring feedback and a defined daily goal.

2.3 Multiplatform Mobile Development

2.3.1 Native versus Cross-Platform Approaches

Mobile application development historically bifurcated into native development—separate codebases per platform using Swift or Objective-C for iOS and Java or Kotlin for Android—and cross-platform development, where a single codebase targets multiple platforms through an abstraction layer. Native development maximises access to platform APIs and delivers the highest performance, but doubles the maintenance burden for teams targeting both major mobile platforms. Cross-platform frameworks trade some low-level control for significantly reduced duplication.

Established cross-platform frameworks include React Native, Flutter, and Kotlin Multiplatform. React Native renders components via a JavaScript bridge, which introduces latency for complex interactions. Flutter compiles to native ARM code and provides its own rendering engine, achieving near-native performance but diverging visually from platform conventions. Kotlin Multiplatform takes a different approach: it shares only business logic across platforms while delegating the user interface to platform-native or declarative frameworks, enabling a balance between code sharing and native look and feel.

2.3.2 Justification for Kotlin Multiplatform and Compose Multiplatform

U!REKA Play4Change adopts Kotlin Multiplatform for shared business logic—including the scoring engine, data models, network clients, and local caching—combined with Compose Multiplatform for the shared declarative user interface. This combination allows Android and iOS to share both logic and UI components from a single codebase while maintaining access to native APIs where required. The choice reduces duplication and accelerates feature delivery without sacrificing the performance characteristics required for smooth animated game interfaces.

2.4 AI-Generated Content in Education

2.4.1 Large Language Models for Educational Content

Large language models (LLMs) have demonstrated strong capability in generating coherent, contextually appropriate text across a wide range of domains. Their application to educational content generation is a rapidly growing area: LLMs can produce question-and-answer pairs, explanatory summaries, and scenario-based tasks at scale and at low marginal cost compared to human authoring. The primary challenges are factual accuracy, content diversity, and deduplication, all of which require careful pipeline design.

2.4.2 LangChain4j and the Mistral API

LangChain4j is a Java library that provides abstractions for building LLM-powered applications, including prompt templating, model chaining, and output parsing. It supports multiple model backends, including the Mistral API, which provides instruction-tuned models well suited to structured task generation [3, 4]. U!REKA Play4Change uses LangChain4j to orchestrate nightly batch jobs that generate

daily tasks parameterised by subject domain and difficulty level. Generated tasks are embedded using a sentence-level embedding model and compared against existing stored embeddings via pgvector to detect and discard near-duplicate content before persistence [5].

2.5 Related Platforms

Several gamified learning platforms have gained widespread adoption, including Duolingo, Kahoot, and Quizlet. Duolingo applies a streak-based engagement model to language learning and demonstrates strong retention at scale; however, its content is entirely hand-authored by domain specialists, limiting expansion to new topics. Kahoot focuses on synchronous, instructor-led quiz sessions and is therefore unsuitable for self-paced asynchronous learning. Quizlet allows community-generated flashcard decks but does not provide an adaptive scoring engine or AI-generated content.

None of these platforms combines multiplatform delivery from a single codebase with automated AI-driven content generation. U!REKA Play4Change addresses this gap by integrating both capabilities, enabling educational networks to deploy a fully configured, self-sustaining gamified platform for any subject domain without sustained content authoring effort.

Capítulo 3

Tutorial de LaTeX com exemplos

Este capítulo irá mostrar como pode inserir tabelas, figuras e/ou listagens. Considere-se que listagens não são todo o código mas partes do código. Estas irão suportar algumas frases dos parágrafos escritos. Não se esqueça, que todas as tabelas, figuras e listagens devem ser referenciadas no corpo do texto.

3.1 Tabelas

Some notes are important to followed, such as present in Tabela 3.1: i) Not defined vertical lines; ii) The legend must be on top; iii) Use `\toprule`, `\midrule` and `\bottomrule` to draw horizontal lines.

Tabela 3.1: Table's rules.

Item		
Animal	Description	Price ($)
Gnat	per gram	13.65
	each	0.01
Gnu	stuffed	92.50
Emu	stuffed	33.33

3.2 Importar Figuras

3.2.1 Inserir figuras lado-a-lado com texto

You should only use this feature is really necessary. This means, you have a very small image, that will look lonely just with text above and below.

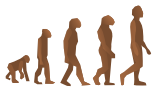


Figura 3.1: Vectorial image

`\usepackage{wrapfig}`

This then gives you access to:

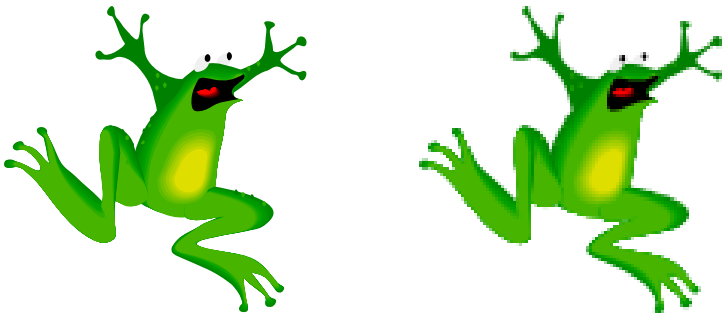
`\begin{wrapfigure}[lineheight]{alignment}{width}`

Alignment can normally be either 'l' for left, or 'r' for right. Lowercase 'l' or 'r' forces the figure to start precisely where specified (and may cause it to run over page breaks), while capital 'L' or 'R' allows the figure to float. If you defined your

document as twosided, the alignment can also be 'i' for inside or 'o' for outside, as well as 'I' or 'O'. The width is obviously the width of the figure.

3.2.2 Outras imagens

There are two different ways to place two figures/tables side-by-side. More complicated figures with multiple images. You can do this using subfigure environments inside a figure environment. Subfigure will alphabetically number your subfigures and you have access to the complete reference as usual through `\ref{fig:figurelabel}`, Figura 3.2, or Figura 3.2b using `\ref{fig:subfigurelabel}`.



(a) (b)

Figura 3.2: Subfigure example with vectorial and no-vectorial images

3.3 Listagens

Using the package listings you can add non-formatted text as you would do with `\begin{verbatim}` but its main aim is to include the source code of any programming language within your document, or inline with `\codejava{int i;}`. If you wish to include pseudocode or algorithms see http://en.wikibooks.org/wiki/LaTeX/Algorithms_and_Pseudocode, as Listagem 3.1.

```
public class Calculadora{
    char c1 = 'c', c2 = 'f';
    System.out.println("The sum is " + (f1+f2));
    System.out.println("The mod is " + (n1%n2));
    System.out.println("The increment is " + (f1+ ++f2) + " e " + (++n1));
  }}
```

Listagem 3.1: Operações elementares de uma Calculadora

Written by Matilde Pós-de-Mina Pato[1],
2020 September - version 2.2

[1]O autor escreve segundo o antigo AO

Referências

[1] Sebastian Deterding, Dan Dixon, Rilla Khaled & Lennart Nacke, "From Game Design Elements to Gamefulness: Defining Gamification", em *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, sér. MindTrek '11, New York, NY, USA: ACM, 2011, páginas 9–15.

[2] Juho Hamari, Jonna Koivisto & Harri Sarsa, "Does Gamification Work? – A Literature Review of Empirical Studies on Gamification", em *Proceedings of the 47th Hawaii International Conference on System Sciences*, sér. HICSS '14, Washington, DC, USA: IEEE, 2014, páginas 3025–3034.

[3] LangChain4j Contributors, *LangChain4j – Java Implementation of LangChain*, https://langchain4j.dev, Accessed: March 2026.

[4] Mistral AI, *Mistral AI Models*, https://mistral.ai, Accessed: March 2026.

[5] pgvector Contributors, *pgvector: Open-Source Vector Similarity Search for Postgres*, https://github.com/pgvector/pgvector, Accessed: March 2026.

[6] Yu Zhonggen, "A Meta-Analysis of Use of Serious Games in Education over a Decade", *International Journal of Computer Games Technology*, vol. 2019, páginas 1–8, 2019.

Apêndice A

Applied Survival Analysis by Hosmer and Lemeshow

Stata Textbook Examples Applied Survival Analysis by Hosmer and Lemeshow [**newtest**]

The data files used for the examples in this text can be downloaded in a zip file from the Wiley FTP website or the Stata Web site.

```
# The R package(s) needed for this chapter is the survival package.
# We currently use R 2.0.1 patched version. You may want to make sure
# that packages on your local machine are up to date. You can perform
# updating in R using update.packages() function.

# url: http://www.ats.ucla.edu/stat/r/examples/
# data set is hmohiv.csv.
hmohiv<-read.table("http://www.ats.ucla.edu/stat/r/examples/asa/hmohiv.csv", sep=",",
    header = TRUE)
attach(hmohiv)
hmohiv

# using the hmohiv data set. To control the type of symbol, a variable called psymbol is created.
# It takes value 1 and 2, so the symbol type will be 1 and 2.
psymbol<-censor+1
table(psymbol)

plot(age, time, pch=(psymbol))
legend(40, 60, c("Censor=1", "Censor=0"), pch=(psymbol))

age1<-1000/age
plot(age1, time, pch=(psymbol))
legend(40, 30, c("Censor=1", "Censor=0"), pch=(psymbol))

# Package "survival" is needed for this analysis and for most of the analyses in the book.
library(survival)
test <- survreg( Surv(time, censor) ~ age, dist="exponential")
```

```
summary(test)

pred <- predict(test, type="response")
ord<-order(age)
age_ord<-age[ord]
pred_ord<-pred[ord]
plot(age, time, pch=(psymbol))
lines(age_ord, pred_ord)
legend(40, 60, c("Censor=1", "Censor=0"), pch=(psymbol))
```